

Advanced Certificate Programme in Agentic AI & RAG Engineering

Naive RAG from Scratch

Phase 2 begins · Build the engine before the framework



Phase 2 Starts Here

PHASE 1 • DONE

The platform

- FastAPI service with /ask + /ask_batched
- Tool-calling Answer with cost + confidence
- 20-entry golden set + LLM-as-judge eval
- ADR v1 locked · M1 milestone shipped

PHASE 2 • STARTS NOW

The engine

- **W6 — Naive RAG from scratch ← today**
- W7 — Better chunking, embeddings, rerank
- W8-W10 — Vector DBs, hybrid, metadata filtering
- W11 — Frameworks (LangChain / LlamaIndex)
- W12 — DR #2, production-ready RAG

By W12 you'll have a production-ready RAG. By the end of today, you'll have built it once — from scratch.

What You'll Build this Week

A complete RAG system in ~100 lines of raw Python

No LangChain. No LlamaIndex. No vector DB. Just Python, numpy, and the OpenAI APIs.



By Friday you'll have:

- data/embeddings.json — your corpus embedded and cached
- src/rag/naive_rag.py — the working pipeline
- docs/kpi/wk6-snapshot.md — your first KPI snapshot
- An updated ADR with baseline numbers + the 3 biggest naive-RAG limits

Today's Roadmap

01

Why RAG matters (and when it doesn't)

Decision rule. 3 wins, 3 doesn'ts.

25 min

03

Build naive RAG together (live)

~100 lines, blank file → working system.

60 min

02

RAG anatomy on the whiteboard

Full pipeline + failure mode at every stage.

25 min

04

KPI reporting begins · first snapshot

Locked template. Weekly habit from now.

20 min

15-min break between Topic 2 and Topic 3. The 60-min hands-on is the longest in the programme so far.

By the End of this Week, You Can...

O1

Articulate when RAG is the right architecture — and when it isn't (long-context, fine-tuning, simple search).

O2

Build a complete naive RAG in raw Python: embed, store, similarity-search, construct prompt, generate answer.

O3

Apply your naive RAG to your real capstone corpus and run it against the W5 golden set.

O4

Compute and report your first capstone KPI snapshot — cost/query, latency, retrieval hit rate, grounded response rate.

O5

Identify the limits of naive RAG (poor chunking, no rerank, no metadata filtering) and link them to W7-W11.

O2 is what you build live. O3-O4 is your lab. O5 is the seed for W7-W11.

Why RAG Matters

(and when it doesn't)

A decision rule, not a religion.

The Problem RAG Solves

WITHOUT RAG

The model knows what it knew in training.

- Knowledge frozen at the cutoff date
- No access to your proprietary documents
- Hallucinates when asked outside its training set
- Can't cite sources — you can't verify

WITH RAG

The model reads YOUR documents at query time.

- Knowledge as fresh as the documents
- Reads your wiki / policies / docs directly
- Grounded — answers tied to retrieved chunks
- Sources travel with the answer — auditable

***Reach for RAG when knowledge updates faster than you can retrain,
and your context is too big to fit in one prompt.***

When RAG Wins



Fast-changing corpus

Internal wiki updated weekly · policies that revise · product docs that ship every sprint

Retraining can't keep up. Long-context blows the budget. RAG retrieves the latest at query time.



Large, structured knowledge base

10K pages of policy · 100K tickets · entire codebase · regulatory archive

Won't fit in any context window. Most queries need just 3–5 relevant chunks.



Citations matter

Regulated industries · legal · medical · finance · audit trails

Sources must travel with the answer. RAG is the only architecture that naturally provides them.

When RAG Doesn't



Small, stable corpus

Product manual under 50 pages · stable FAQ · documentation that changes quarterly

Just paste it into the system prompt. Long-context wins: simpler, faster, cheaper.



Style or skill transfer

Brand voice · domain-specific reasoning · code generation in your stack's idiom

Fine-tuning teaches the model a behaviour. RAG can't change how the model thinks.



Structured, exact-match data

Order status · account balance · 'when does my flight depart'

That's a database query, not a similarity search. Use SQL, return the row, format the answer.

The Decision Rule

Reach for RAG when...

**knowledge updates faster than you can retrain,
and context is too big to fit in one prompt.**

If that's not you, consider...

Long-context

Small corpus. Paste it. GPT-4o handles 128K tokens for cheap.

Fine-tuning

Stable style or skill. ~\$5-50 of training data, lasts months.

Search-then-answer

Structured FAQ. Keyword/BM25 hit → templated response.

RAG vs Long-context vs Fine-tuning

	RAG	Long-context	Fine-tuning
Best when	knowledge changes	corpus is small	behaviour changes
Setup cost	low — index once	zero	high — data + training
Per-query cost	tokens for top-K chunks	tokens for full corpus	same as base model
Latency	+ retrieval step	long prompt = slow	no extra step
Citations	yes — natural	possible if asked	no
Knowledge freshness	as fresh as the index	as fresh as the prompt	as of training
Best example	internal wiki Q&A	single-doc summarise	brand-voice writer

Topic 1 — Takeaway

RAG is not magic.

It's a useful pattern for specific problems.

Phase 2 is about making this pattern good.

By W12 you'll know the trade-offs cold.

NEXT

Topic 2 — RAG anatomy on the whiteboard

RAG Anatomy

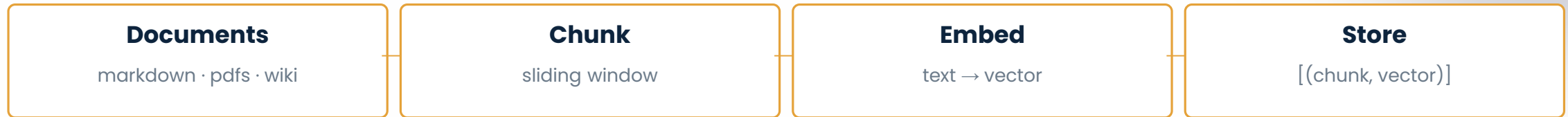
on the Whiteboard

Every stage has a KPI. Every stage can fail. Learn the map before you build.

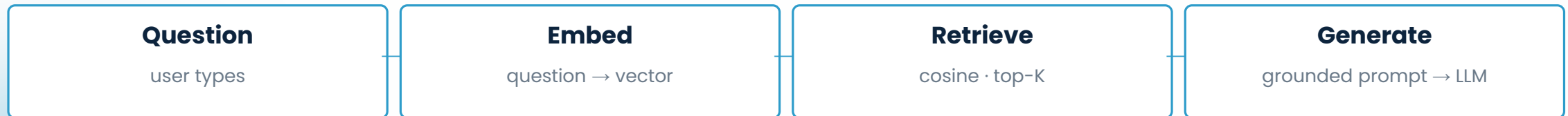
The Full Pipeline

Two passes — INDEX TIME (once) and QUERY TIME (every question)

INDEX TIME · RUNS ONCE



QUERY TIME · EVERY QUESTION



Both passes share the same store.

Memorise this shape. Phase 2 (W7-W11) is making each box better — one at a time.

W7 better chunking · W8 vector DBs · W9 hybrid retrieval · W10 metadata · W11 frameworks.

Stage 1 - Index Time

Documents

What goes in is what comes out. The cheapest, highest-leverage improvement is your corpus.

What it does

- Source of truth: markdown, PDFs, wiki pages.
- 10-20 documents is typical for a single capstone.
- Cleaned: no nav cruft, no boilerplate, no duplicates.
- Pre-chunk: keep paragraphs intact where possible.

What can fail

- Stale documents (knowledge that's wrong now).
- PDFs with multi-column layout — extraction garbles them.
- Mixed languages, mixed formats, mixed audiences.
- Documents that don't actually answer your golden questions.

What you measure

- Document count.
- Total tokens ($\approx$ index size).
- Coverage — % of golden questions answerable from corpus.
- Freshness — last-modified spread.

If your golden set asks about expense policy but your corpus has only leave policy — your RAG can't win.

Stage 2 - Index Time

Chunking

Why chunk at all? Two reasons: embedding model size limits, and you want to retrieve the relevant **part** of a doc, not the whole doc.

Common strategies

Sliding window

Fixed N chars with overlap.

Simple. Cuts mid-sentence.

By sentence

Split on . ! ?

Cleaner boundaries. Variable sizes.

By section

Split on headings

Coherent. Some sections too big.

Semantic

Split where topic changes

Best signal. Most complex to build.

W6 • WE USE

Sliding window. 500 chars. 50 char overlap.

Naive on purpose. You'll SEE where it breaks in the lab — that's the lesson.

What can fail

- Cuts mid-sentence → context lost across chunks
- Too small (200 chars) → noise dominates similarity
- Too large (2000 chars) → wastes context budget
- No overlap → important info straddles boundary

Stage 3 - Index Time

Embeddings

Text → vector of floats. Similar meaning → similar vectors. The magic that lets us compare ideas.

"Employees get 20 days of leave per year."



[0.012, -0.84, 0.31, ..., 0.07] # 1536 dims

W6 · DEFAULT

OpenAI · text-embedding-3-small

1536 dimensions · \$0.02 per 1M tokens · API call

~\$0.01 for the entire W6 lab. Fast. Good enough for everything we'll do here.

OPTIONAL · LOCAL

sentence-transformers · all-MiniLM-L6-v2

384 dimensions · Free · Runs locally · CPU OK

For cost-sensitive learners or privacy-sensitive corpora. Slower per query.

What can fail

- Model trained on a different domain (e.g. medical text vs your policy docs)
- Vectors not normalised (some pipelines forget — cosine similarity goes wrong)
- Embed at index time but cache lost / re-embed on every query = expensive

Stage 4 - Index Time

Vector Store

Just a list. This week. Vector DBs land in W8 — once you've felt the pain of doing without one.

```
store = [  
  {"chunk": "Employees get 20 days...",  
   "embedding": [0.012, -0.84, ...], # 1536 dims  
   "source": "leave_policy.md"},  
  {"chunk": "Reimbursements over $50...",  
   "embedding": [-0.21, 0.55, ...],  
   "source": "expense_policy.md"},  
  # ... one entry per chunk × all documents  
]
```

What can fail

- JSON file gets stale — corpus updated but embeddings not rebuilt
- Linear scan becomes slow once you exceed ~10K chunks (this is what W8 fixes)
- Server restart loses store if you didn't persist (oops)

Persistence

- **Save to data/embeddings.json once**

build_index.py · re-runs only when corpus changes

- **Load on every server start**

$O(1)$ — just read the file

- **Cost transparent**

Re-running build_index re-embeds. Cache prevents this.

Stage 5 - Query Time

Retrieval

Embed the question. Compare against every chunk. Take the top-K. Numpy will do this in one line.

Cosine similarity — the intuition

Angle between two vectors.

1.0 = same direction (same meaning)

0.0 = perpendicular (unrelated)

-1.0 = opposite (rare in text embeddings)

Real-world question-vs-chunk scores: typically 0.25–0.85.

We don't filter on absolute score — we rank.

Top-K — the trade-off

K = 1 → fast, narrow. Misses if the best chunk is #2.

K = 3 → W6 default. Usually enough for capstone Q&A.

K = 10 → more recall, more tokens, more noise.

K = 20 → context window blowing up — and the LLM gets confused.

Higher K isn't always better. Measure it.

What can fail

- Top-K is right semantically but wrong document (a 'leave' chunk for an expense question)
- Forgot to normalise vectors — cosine on un-normed embeddings gives weird ranking
- Embedding model captured surface form, not meaning — synonyms miss each other
- K too low — answer chunk is at rank 5, but we only retrieved 3

Stage 6 - Query Time

Generation

Drop the retrieved chunks into a prompt. Tell the model to ground its answer in them. Return the answer + the source chunks.

The grounded-answer prompt

You are an assistant answering questions using the provided context.

Context:

[Chunk 1] Employees get 20 days of paid leave per year.
[Chunk 2] Annual leave accrues monthly, prorated by start date.
[Chunk 3] Unused leave does not roll over past December 31.

Question: How many days of leave do I get?

Rules:

- Answer only from the context above.
- If the context does not answer, say "I don't have enough information."
- Cite the chunk number(s) you used.

What this template gives you

(1) Grounded answers — the LLM has the facts in-context

(2) Refusals when retrieval missed — better than hallucination

(3) Source citations — auditable, debuggable

Build Naive RAG

Together · Live coding

~100 lines of raw Python. Blank file → working system. No frameworks. We build, we run, we see it work.

What we'll Retrieve From

Five short policy documents for an Enterprise Knowledge Assistant

Real-feeling content. 200–400 words each. Themed to the W5 golden set so questions actually have answers.

POLICY · MD	POLICY · MD	POLICY · MD	POLICY · MD	POLICY · MD
leave_policy.md Annual leave, sick days, accrual, carry-over rules.	expense_policy.md Reimbursement caps, receipt requirements, approval flow.	wfh_policy.md Remote work eligibility, equipment, location restrictions.	byod_policy.md Personal device usage, security requirements, MDM.	dress_code.md Office attire, client-facing days, casual Fridays.

We chose this theme because your W5 golden set already contains questions for it.

Same questions. Different answer engine. You'll feel the upgrade.

The 5 Functions

One blank file. Five functions. ~100 lines. Plenty of whitespace.

1

`chunk_text(text, size=500, overlap=50)`

Sliding window over a document. Returns list[str].

~10 lines

2

`embed(text)`

One OpenAI call. Returns list[float] of length 1536.

~6 lines

3

`build_store(docs)`

For each doc → chunk → embed → assemble store. Cache to JSON.

~15 lines

4

`retrieve(question, store, k=3)`

Embed question. Cosine vs all chunks. argsort. Return top-K.

~10 lines

5

`ask_rag(question, store)`

Retrieve. Build grounded prompt. Call LLM. Return answer + sources.

~20 lines

Helpers + imports + a couple of constants gets us to ~100 lines. '100 lines' is a target, not a constraint.

FUNCTION 1 / 5

chunk_text — sliding window

```
def chunk_text(text: str, size: int = 500, overlap: int = 50) -> list[str]:  
    """Sliding window. Cuts mid-sentence on purpose."""  
    chunks = []  
    step = size - overlap  
    for start in range(0, len(text), step):  
        chunk = text[start:start + size]  
        if chunk.strip():  
            chunks.append(chunk)  
    return chunks
```

What's happening

- **step = size - overlap**

Move forward 450 chars each iteration — leaves 50 of overlap.

- **chunk.strip()**

Skip whitespace-only chunks (file end, blank pages).

- **Returns plain strings**

No metadata yet. We'll bundle metadata at build_store time.

What can go wrong with this naive chunker

Sentences cut in half. Headings stranded from their body. Two unrelated topics in one chunk. → W7 fixes this.

FUNCTION 2 / 5

embed — text → vector

```
from openai import OpenAI
client = OpenAI()

def embed(text: str) -> list[float]:
    """One vector for one string. text-embedding-3-small."""
    resp = client.embeddings.create(
        model="text-embedding-3-small",
        input=text,
    )
    return resp.data[0].embedding
```

What this costs

\$0.02 per 1M tokens.

5 docs × 4 chunks × ~120 tokens = 2,400 tokens

= \$0.00005 to embed everything

Plus one query embedding per question — same rate.

Whole W6 lab: \$0.05–0.20 per learner depending on corpus size.

Why we cache

Re-embedding the same corpus on every server restart is wasteful — and unbounded as your corpus grows.

We embed once, save to data/embeddings.json, load on startup.

build_index.py is a one-shot — re-run it when docs change.

FUNCTION 3 / 5

build_store — assemble the index

```
def build_store(docs: dict[str, str]) -> list[dict]:  
    """docs = {filename: full_text}. Returns list[chunk, embedding, source]."""  
    store = []  
    for filename, text in docs.items():  
        for chunk in chunk_text(text):  
            store.append({  
                "chunk": chunk,  
                "embedding": embed(chunk),  
                "source": filename,  
            })  
    return store
```

Notes

- Dict-per-chunk — easy to extend later (timestamp, page, section).
- Each chunk carries its source filename — that's what we cite back.
- Sequential embed calls. Fine for 5 docs; we'd batch for 5,000.
- Persist with `json.dump(...)` → `data/embeddings.json`. Load with `json.load`.
- Re-run `build_store` only when corpus changes (`build_index.py` wraps this).

FUNCTION 4 / 5

retrieve — cosine + top-K

```
import numpy as np

def retrieve(question: str, store: list, k: int = 3):
    """Embed question, score against every chunk, return top-K rows."""
    q_vec = np.array(embed(question))

    embeddings = np.array([row["embedding"] for row in store])

    # Cosine similarity – vectors come pre-normalised by OpenAI
    scores = embeddings @ q_vec

    top_idx = np.argsort(scores)[::-1][:k]

    return [store[i] | {"score": float(scores[i])} for i in top_idx]
```

Key bits

- `embeddings @ q_vec`

Matrix-vector dot product = N cosine scores in one line.

- `argsort()[::-1][:k]`

Sort ascending, reverse, take first K = top-K indices.

- `dict | {...}`

Spread + add 'score' to each returned row (Python 3.9+).

- **Returns full rows**

chunk + embedding + source + score. Caller picks what it needs.

FUNCTION 5 / 5 - The Orchestrator

ask_rag — put it all together

```
def ask_rag(question: str, store, k=3):  
    """Retrieve top-K chunks. Build grounded prompt. Generate answer."""  
    top = retrieve(question, store, k=k)  
  
    context = "\n\n" join  
        f"[Chunk {i+1}] {row['source']}\n{row['chunk']}"  
        for i, row in enumerate(top)  
    )  
  
    prompt = f"""Answer the question using ONLY the context.  
Context:\n{context}\n\nQuestion: {question}\n\nAnswer: """""  
  
    response = client.chat.completions.create(  
        model="gpt-4o-mini", messages=[{"role": "user", "content": prompt}])  
    return {"answer": response.choices[0].message.content,  
            "sources": [r["source"] for r in top]}
```

What you call

```
result = ask_rag(  
    "How many leave days  
do I get?", store)
```

What you get back

'answer':

*"Employees get 20 days of paid leave per year,
accrued monthly."*

'sources':

*['leave_policy.md', 'leave_policy.md',
'leave_policy.md']*

Live build • now

Open a blank file. Follow along. Type with me.

Pair if you'd rather watch than type.

Junior + senior pair recommended. One drives, one navigates. Switch at function boundaries.

If you fall behind, just keep up with the running file in the lab. We don't backtrack.

KPI Reporting

Begins this week · Weekly habit forever

Every lab from now on ends with a KPI snapshot. Same template every week through W30.

The 8 KPIs

Not all 8 are measurable every week. The format stays the same — we fill N/A when we can't.

KPI	What it measures	Measurable from
Task success rate	% of golden questions answered acceptably (eyeball at W6, judge from W7+)	W6 · eyeball
Grounded response rate	Of answers given, % that cite retrieved chunks (LLM-as-judge)	W6 · judge
Retrieval hit rate	Of golden questions, % where top-K contains the answer's source doc	W6
Cost / query	Sum of embedding + chat tokens × pricing, per question	W6
Latency (p50 / p95)	End-to-end seconds, embedded + retrieve + generate	W6
Tool / function success	RAG-as-tool calls that completed without error (multi-step agents)	W11+ · N/A here
Escalation rate	% of questions handed off to human (after confidence threshold)	W13+ · N/A here
Stakeholder NPS	Quarterly survey from corpus owners and end users	Post-launch · N/A

What to Measure this Week

Five KPIs you can compute today, one per stage of the pipeline.

Task success rate

2-min eyeball: read each of the 20 answers, mark Y/N/Partial.

X / 20 acceptable

Grounded response rate

Re-use the W4 LLM-as-judge — runs against your rag_runs.

X%

Retrieval hit rate

Of 20 questions, how many returned a chunk from the right source doc?

X / 20

Cost / query

Sum of token costs / 20. Embedding + chat. Already logged in rag_runs.

\$0.0XX

Latency p50 / p95

Time taken for ask_rag() per question. Numpy percentile.

Xs / Ys

The Locked Snapshot Template

This format stays identical from W6 to W30. `compute_kpis.py` fills it in for you.

`docs/kpi/wk6-snapshot.md`

KPI Snapshot – Week 6 – Naive RAG baseline

KPI	Value	How measured	Notes
-----	-----	-----	-----
Task success rate	14 / 20	Eyeball, see notes below	3 N (wrong doc)
Grounded response rate	92%	LLM-as-judge (gpt-4o)	1 hallucination
Retrieval hit rate	18 / 20	Source doc in top-3	2 misses on edges
Cost / query	\$0.0021	Sum of token costs / 20	Embedding dominates
Latency p50 / p95	1.2s / 1.8s	time per ask_rag call	Mostly LLM call
Tool / function success	N/A	not applicable yet (W11+)	
Escalation rate	N/A	not applicable yet (W13+)	
Stakeholder NPS	N/A	not applicable yet	

Eight rows. Every week. Even when half say N/A. Continuity matters more than completeness.

What "good" Looks Like

Honest baselines — naive RAG isn't great yet. That's the point.

KPI	Acceptable	Good	Reading
Task success rate	≥ 12 / 20	≥ 16 / 20	<i>Below 10 → corpus or chunking problem.</i>
Grounded resp. rate	≥ 80%	≥ 95%	<i>Low means LLM ignores context.</i>
Retrieval hit rate	≥ 14 / 20	≥ 18 / 20	<i>Low = chunking and embeddings.</i>
Cost / query	< \$0.01	< \$0.005	<i>Mostly embedding spend at W6.</i>
Latency p95	< 3s	< 1.5s	<i>Most of it is the LLM call.</i>

If your numbers are wildly below "Acceptable" — check your corpus first, then your chunker.

Takeaway

Measure before you tune.

Every Phase 2 change you make from here, you justify with KPI movement.

W6 numbers are your baseline.

W7-W11 only ship if they move the right numbers in the right direction.

NEXT

What's wrong with what we just built?

What's Wrong with Naive RAG?

Each limit you'll feel is a future week's lesson. The map of the next 5 weeks.



Chunking is dumb

Cuts mid-sentence. No structure awareness. No metadata.

W7 — Better chunking



One model fits all

text-embedding-3-small is generic. Domain-specific embeddings exist.

W7 — Embedding choice



Linear scan won't scale

Fine for 100 chunks. Painful at 10K. Useless at 1M.

W8 — Vector databases



Semantic only

Keyword 'CEO-2024-Q3' doesn't embed well. Hybrid wins.

W9 — Hybrid retrieval



No metadata filtering

Can't say 'only HR docs from this quarter'. Often needed.

W10 — Metadata + rerank

W11 = frameworks (LangChain / LlamaIndex). W12 = DR #2 and production-ready RAG.

Your lab — 4 steps

Step 1 · 45 min

Apply naive RAG to YOUR capstone corpus.

Adapt `naive_rag.py` to your 10–20 docs. Re-embed. First end-to-end query on real content.

Step 2 · 60 min

Run the full 20–entry golden set.

All 20 questions → `ask_rag` → save to `rag_runs` table. Eyeball-grade each answer Y/N/Partial.

Step 3 · 45 min

Compute your first KPI snapshot.

Run `compute_kpis.py`. Fill `docs/kpi/wk6-snapshot.md`. This becomes your weekly artifact.

Step 4 · 30 min

Update ADR + identify Top 3 limits.

Add baseline KPIs to ADR 0001. Pick the 3 worst things you noticed — those preview W7–W10.

BY FRIDAY: `data/embeddings.json` · `src/rag/naive_rag.py` · `docs/kpi/wk6-snapshot.md` · ADR update

END OF WEEK 6

You've built it from scratch.

Now the rest of Phase 2 is about making it good.

NEXT WEEK

W7 — Better chunking, embeddings, and a first attempt at rerank

Pre-read: Anthropic — Contextual Retrieval (you should already have read this for W6).

Bring: your Top 3 limits from Step 4 — we'll start picking them off.

Q & A

Thank you